Exam-1 Program Verification 2021/2022

Megaron, 4th Oct 2021, 11:30 - 14:30

Lecturers: Wishnu Prasetya, Anna-Lena Lamprecht

1 Predicate Transformer [3pt]

1. [1 pt]

   (a) Give the weakest pre-condition of the statement below, with respect to the given post-condition. **Show** your wlp calculation; but **do not** simplify the resulting formula so that I can check your calculation.

   $\{* \; ? \; *\}$
   assume $\mathtt{k}{\geq}\mathtt{0} \;\wedge\; \mathtt{y} * \mathtt{x}^{\mathtt{k}} = \alpha^{1001}$
   assume $\mathtt{k} > \mathtt{0}$
   if $\mathtt{k}$ **mod** $\mathtt{2} = \mathtt{0}$ **then** $\{\mathtt{x} := \mathtt{x}{*}\mathtt{x} \;;\; \mathtt{k} := \mathtt{k}/\mathtt{2}\}$
   else $\{\mathtt{y} := \mathtt{x}{*}\mathtt{y} \;;\; \mathtt{k} := \mathtt{k}{-}\mathtt{1}\}$
   $\{* \; \mathtt{k}{\geq}\mathtt{0} \;\wedge\; \mathtt{y} * \mathtt{x}^{\mathtt{k}} = \alpha^{1001} \; *\}$ // post-cond

   Solution: (not showing the calculation)
   let's call the post-condition Q. The if-then-else results in this wlp P_0:

   $$P_0 : \begin{cases} even(k) \Rightarrow k/2{\geq}0 \wedge y(x^2)^{k/2}{=}\alpha^{1001} \\ \wedge \\ \neg even(k) \Rightarrow k{-}1{\geq}0 \wedge yxx^{k-1}{=}\alpha^{1001} \end{cases}$$

   where $even(k)$ is the formula k **mod** $2 = 0$.
   Notice that the first **assume** is the same as Q. So the final wlp will be:

   $wlp: \; Q \Rightarrow (k{>}0 \Rightarrow P_0)$

   This wlp is valid btw.

   (b) Consider the statement below. Give the weakest pre-conditions at position Q_2 and Q_1, with respect to the indicated post-condition. The array `a` can be assumed to be of size `N>0`. Show the wlp calculation. Do reduce/remove all **repby** sub-expressions from your results.
   Be careful with your handling of $\forall$.

   $\{Q_1?\}$ **assume** $(\forall \mathtt{j} : \mathtt{1}{\leq}\mathtt{j}{\leq}\mathtt{N} : \mathtt{a}[\mathtt{j}{-}\mathtt{1}] \geq \mathtt{a}[\mathtt{0}])$
   $\{Q_2?\}$ **assume** $\mathtt{1}{\leq}\mathtt{k}{\leq}\mathtt{N}$;
   $\mathtt{a}[\mathtt{k}] := \mathtt{a}[\mathtt{k}] - \mathtt{1}$;
   $\mathtt{k} := \mathtt{k}{-}\mathtt{1}$
   $\{* \; (\forall \mathtt{j} : \mathtt{1}{\leq}\mathtt{j}{\leq}\mathtt{N} : \mathtt{a}[\mathtt{j}{-}\mathtt{1}] \geq \mathtt{a}[\mathtt{0}]) \; *\}$ // post-cond

   Just to remind you, the notation $(\forall j : P\ j : Q\ j)$ means the same as $(\forall j :: P\ j \;\Rightarrow\; Q\ j)$.
   Solution: Applying wlp-calculation to the given post-condition above we obtain this as Q_2:

   $1{\leq}k{\leq}N$
   $\Rightarrow$
   $(\forall j : 1{\leq}j{\leq}N : a(k \textbf{ repby } a[k]{-}1)[j{-}1] \geq a(k \textbf{ repby } a[k]{-}1)[0])$

When we expand the definition of **repby** we obtain:

$$
\begin{array}{l}
1{\leq}k{\leq}N \\
\Rightarrow \\
(\forall j : 1{\leq}j{\leq}N : (j{-}1{=}k \rightarrow a[k]{-}1 \mid a[j{-}1]) \geq \underbrace{(0{=}k \rightarrow a[k]{-}1 \mid a[0])}_{\text{can be simplified}}
\end{array}
$$

Since $1 \leq k$ implies that k cannot be 0, the marked term above can be simplified to just $a[0]$. So we obtain:

$$1{\leq}k{\leq}N \;\Rightarrow\; (\forall j : 1{\leq}j{\leq}N : (j{-}1{=}k \rightarrow a[k]{-}1 \mid a[j{-}1]) \geq a[0])$$

We now continue to calculate Q_1, obtaining:

$$
\begin{array}{l}
\overbrace{(\forall j : 1{\leq}j{\leq}N : a[j{-}1] \geq a[0])}^{A_0} \\
\Rightarrow (1{\leq}k{\leq}N \\
\quad \Rightarrow \\
\quad \underbrace{(\forall j : 1{\leq}j{\leq}N : (j{-}1{=}k \rightarrow a[k]{-}1 \mid a[j{-}1]) \geq a[0]))}_{\text{can be simplified}}
\end{array}
$$

Now, the assumptions A_0 implies that for all j in the range $1 \leq j \leq N$, $a[j{-}1]$ is guaranteed to be $\geq a[0]$. So the marked formula above can be simplified to just $a[k]{-}1 \geq a[0]$.

2. [0.8 pt] Consider a simple language of statements as in the language GCL from your project. Let us extend this language with a **switch**-statement with the following syntax:

switch e {
  c_1 $\rightarrow$ $stmt_1$
  ...
  c_K $\rightarrow$ $stmt_K$
}

where e is an integer-typed expression, each c_i is a *distinct* integer constant/literal, and $stmt_i$ is a statement. The **switch**-statement first evaluaes e. If the result is equal to c_i, the whole sequence:

$$stmt_i \;;\; stmt_{i+1} \;;\; ... \;;\; stmt_K$$

will be executed. If e does not evaluate to any of the c_i, the whole statement simply does a **skip**.

An example is shown below:

switch `x` {
  `4` $\rightarrow$ `x := x - 2`
  `2` $\rightarrow$ `x := x - 2`
  `0` $\rightarrow$ `y := x`
}

For example, if the switch expression `x` evaluates to 2, then `x := x-2 ; y := x` will be executed (so, both `x, y` end up with the value 0).

Your task: give a formula for constructing the **wlp** of a **switch**-statement.

Solution:

$$
\begin{array}{l}
(e{=}c_0 \;\Rightarrow\; \mathbf{wlp}\ (S_1; ...; S_K)\ Q) \\
... \\
\wedge\ (e{=}c_i \;\Rightarrow\; \mathbf{wlp}\ (S_i; ...; S_K)\ Q) \\
... \\
\wedge\ (e{=}c_K \;\Rightarrow\; \mathbf{wlp}\ S_K\ Q) \\
\wedge\ (e \neq c_0 \wedge ... \wedge e \neq c_K \;\Rightarrow\; Q)
\end{array}
$$

3. [0.6 pt] A tool developer wants to build a program verification tool based on a post-condition transformer named **cp**. So, given a statement S and a post-condition Q,

   $$\mathbf{cp}\ S\ Q$$

   will construct a pre-condition. When given a specification:

   $$\{P\}\ S\ \{Q\}$$

   the tool first invokes **cp** $S\ Q$. Suppose this results in a predicate P'. The tool then checks the validity of $P \Rightarrow P'$. If the implication is valid, the specification $\{P\}\ S\ \{Q\}$ is accepted, and otherwise it is rejected.

   The tool developer considers two variants of **cp**:

   - Variant-1: a **cp** which is sound but *not* complete.
   - Variant-2: a **cp** which is complete but *not* sound.

   Questions:

   (a) Consider this result of **cp**:

   $$\mathbf{cp}\ \underbrace{(\textbf{if g then x} := \mathtt{x}/2\ \textbf{else x} := \mathtt{x} - 1)}_{\text{statement}}\quad \underbrace{(x \geq 0)}_{\text{post-condition}}$$

   gives:

   $$x/2 \geq 0\ \vee\ x{-}1 \geq 0$$

   Question: which variant of **cp** was used? Explain your answer.

   Answer: **cp** is sound and complete if the following hold (so, if the left and the right terms are equivalent):

   $$\{P\}\ S\ \{Q\} \quad \equiv \quad P \Rightarrow \mathbf{cp}\ S\ Q \text{ is valid}$$

   It is sound if the right term implies the left term. So, if:

   $$P \Rightarrow \mathbf{cp}\ S\ Q \text{ is valid implies that } \{P\}\ S\ \{Q\}$$

   It is complete if the left term implies the right term. So, if:

   $$\{P\}\ S\ \{Q\} \text{ implies that } P \Rightarrow \mathbf{cp}\ S\ Q \text{ is valid}$$

   First notice that the actual wlp is:

   $$(1)\ (g \wedge x/2 \geq 0)\ \vee\ (\neg g \wedge x{-}1 \geq 0)$$

So, the pre-condition produced by **cp** is weaker than the actual wlp. Suppose now a specification $\{P\}\ S\ \{Q\}$ is valid. Then $P \Rightarrow$ **wlp** $S\ Q$ is also valid. Since **cp** $S\ Q$ is weaker than **wlp** $S\ Q$, then P will also imply **cp** $S\ Q$ (that is, $P \Rightarrow$ **cp** $S\ Q$ is also valid). So, we know that **cp** is complete.

cp is *not* sound, because we said that the complete variant is not sound :) But here is a better explanation. Let's take **false** as our guard g in the said if-then-else. Consider $x = 0$ as the given pre-condition P. Notice that $x{=}0 \Rightarrow x/2 \geq 0\ \vee\ x{-}1 \geq 0$ is valid. However, this is *not* a valid specification:

$$\{x{=}0\}\ \ \textbf{if false then}\ \mathtt{x} := \mathtt{x/2}\ \textbf{else}\ \mathtt{x} := \mathtt{x} - \mathtt{1}\ \ \{x \geq 0\}$$

(b) Name one main advantage and one main disadvantage (what could go wrong?) of each variant above.

Solution:

A sound **cp** always produces a valid pre-condition. That is, the following triple is valid: $\{$ **cp** $S\ Q$ $\}\ S\ \{\ Q\ \}$.

Since pre-conditions can be weakened, note that the above implies that if any P that implies **cp** $S\ Q$ will also be a valid pre-condition. So: $\{P\}\ S\ \{Q\}$ is valid. This means that any triple $\{P\}\ S\ \{Q\}$ approved by the tool (when using a sound **cp**), is also guaranteed to be valid.

By contra position this also means that any 'bug' (in the sense of any pre-state s_X that does **not** lead to Q) will be found by the tool, since s_X will not imply **cp** $S\ Q$ either.

Note however, that if the implication $P \Rightarrow$ **cp** $S\ Q$ is not valid, this does not necessarily mean that the specification $\{P\}\ S\ \{Q\}$ is not valid as well. In other words, a 'bug' reported by the tool (a counter example of $\{P\}\ S\ \{Q\}$) may not be a bug in the real program (so, it is a 'false positive').

cp is complete if any valid pre-condition will imply the pre-condition produced by **cp**. So, for all P, if the triple $\{P\}\ S\ \{Q\}$ is valid, then the implication $P \Rightarrow$ **cp** $S\ Q$ is also valid. The reverse does not necessarily hold though. So then the tool, if it uses a complete **cp**, proves that $P \Rightarrow$ **cp** $S\ Q$ is valid, this does not necessarily mean that $\{P\}\ S\ \{Q\}$ is valid.

Note that by cotra-position the above also implies that if the tool discovers that the implication $P \Rightarrow$ **cp** $S\ Q$ is **not** valid (so, it finds a counter example of the implication), the specification $\{P\}\ S\ \{Q\}$ is also not valid, and moreover the found counter example will trigger a real bug in the program.

On the other hand, the tool may not find all bugs.

4. [0.6 pt] Consider the following program, with the given post-condition. The array `a` is of size `N`. The programmer wants to verify that the program will not crash because it tries to access `a` at an invalid index, so he/she adds the `assert` statement in the loop.

$\{*\ ?\ *\}$

while `k<N` **do** {
  assert $\mathtt{0} \leq \mathtt{k} < \mathtt{N}$;
  `sum` := `sum+a[k]` ;
  $\mathtt{k} := \mathtt{k} + \mathtt{1}$ }

$\{*\ \mathtt{0} \leq \mathtt{k}\ *\}$

Show how you can calculate the **wlp** of this loop using fix point (FP) iterations, with respect to the given post-condition. Do the iterations terminate?

Hint: this property is obvious, but perhaps it is helpful to mention anyway: $(p \wedge g) \vee (p \wedge \neg g)$ is equivalent to just p.

Solution:

We calculate the wlp by successively calculating an approximation W_i, as follows:

$$w_0 = \textbf{true}$$

$$W_{i+1} \;=\; (\neg g \wedge Q) \;\vee\; (g \wedge \textbf{wlp}\ body\ W_i)$$

Using this we obtain W_1:

$$W_1 \;=\; (k \geq N \wedge 0 \leq k) \;\vee\; (k < N \wedge 0 \leq k < N)$$

Notice that this can be simplified to $W_1 = 0 \leq k$.

Next, we calculate W_2:

$$W_1 \;=\; (k \geq N \wedge 0 \leq k) \;\vee\; \underbrace{(k < N \wedge 0 \leq k < N \wedge 0 \leq k+1)}_{\text{simplfies to } k<N \wedge 0 \leq k < N}$$

Since $0 \leq k{+}1$ is implied by $0 \leq k$, the second conjunct above simplifies to just $k < N \wedge 0 \leq k < N$, which is the same as in W_1. So, we have:

$$W_1 \;=\; (k \geq N \wedge 0 \leq k) \;\vee\; (k < N \wedge 0 \leq k < N)$$

which again can be simplified to $0 \leq k$. So we have reached a fix point.

2 Dealing with Loop [2 pt]

5. [1.5pt]Below are several small programs. They are valid with respect to the given pre- and post-conditions. For each program, give a **loop invariant** that would be good enough to prove the program's specification. You do not have to deliver a proof of this, but do consider your candidates carefully (e.g. simulate few runs to check them).

   (a) The following program calculates 2^{10}:

   $\{* \texttt{k=0} \wedge \texttt{x=1} *\}$

   while $\texttt{k<10}$ **do** $\{\texttt{k} := \texttt{k+1}\ ;\ \texttt{x} := 2 * \texttt{x}\ \}$

   $\{* \texttt{x} = 2^{\texttt{10}} *\}$

   Answer: Inv: $0 \leq k \leq 10 \;\wedge\; x = 2^k$

   (b) The following program calculates the minimum element of an array. The notation $\#\texttt{a}$ denotes the length of the array a.

   $\{* \#\texttt{a} > 0 *\}$

   $\texttt{m} := \texttt{a}[0]$;
   $\texttt{k} := 1$;
   while $\texttt{k}<\#\texttt{a}$ **do** {
   if $\texttt{a}[\texttt{k}]<\texttt{m}$ **then** $\texttt{m} := \texttt{a}[\texttt{k}]$ **else skip** ;
   $\texttt{k} := \texttt{k+1}$}

   $\{* (\forall \texttt{i} : 0 \leq \texttt{i} < \texttt{k} \# \texttt{a} : \texttt{m} \leq \texttt{a}[\texttt{i}]) *\}$

   Answer: Inv: $1 \leq k \leq \#a \;\wedge\; (\forall i : 0 \leq i < k : m \leq a[i])$

(c) Below is a fragment of the Quicksort algorithm. The program takes the last element of the array a as a so-called *pivot* value. It then re-arranges the array a so that all elements $\leq pivot$ will be placed in the lower half of the array (and all elements $> pivot$ are thus placed in the array's upper half). The program $\texttt{swap}(a, i, k)$ swaps the i-th and k-th element of the array a.

$\{* \texttt{N}=\#\texttt{a} \wedge \texttt{N}>0 *\}$

$\texttt{pivot} := \texttt{a}[\texttt{N}-1]$;
$\texttt{i} := 0$; $\texttt{k} := 0$;
while $\texttt{k}<\texttt{N}$ **do** {
  if $\texttt{a}[\texttt{k}] \leq \texttt{pivot}$
    then $\texttt{swap}(\texttt{a}, \texttt{i}, \texttt{k})$; $\texttt{i} := \texttt{i}+1$
    else skip ;
  $\texttt{k} := \texttt{k}+1$}

$\{* (\forall \texttt{v} : 0 \leq \texttt{v} < \texttt{i} : \texttt{a}[\texttt{v}] \leq \texttt{pivot}) \wedge (\forall \texttt{v} : \texttt{i} \leq \texttt{v} < \texttt{N} : \texttt{pivot} < \texttt{a}[\texttt{v}]) *\}$

Answer: Inv: $0 \leq i \leq k \leq N \wedge (\forall v : 0 \leq v < i : a[v] \leq pivot) \wedge (\forall v : i \leq v < k : pivot < a[v])$

6. [0.5pt] Consider the GCL language as in your project. Suppose we extend this language with a repeat-loop:

   inv I **repeat** S **until** g

   This statement will repeatedly do S until the condition g becomes true. Notice that S will thus be executed at least once. I is a loop-invariant that the programmer can annotate.

   Suppose the programmer does provide an invariant, give a rule to calculate the **wlp** of the above **repeat**-construct. Do mention under which conditions your proposed wlp is sound.

   Answer: We can define:

   wlp (**inv** I **repeat** S **until** g) Q = **wlp** (S ; **inv** I **while** $\neg g$ **do**S) Q

   where **wlp** of **inv** I **while** g **do** S with respect to Q is just I, provided the following hold/valid:

   (a) $I \wedge g \Rightarrow Q$
   (b) $I \wedge \neg g \Rightarrow$ **wlp** S I

3 Verification of Temporal Properties [5 pt]

7. [1pt] Consider a concurrent system of N processes $P_1 ... P_N$, with $N \geq 3$. They work together using three resources r, s, and t. Each resource has the corresponding lock, e.g. $r.lock$ is the lock of r.

   When a process P_i wants to use r, it first indicates this by setting a flag $req(P_i, r)$ to true. The process can use r once it has r's lock. Let's also introduce the state predicate $has(P_i, r.lock)$ that is true if and only if P has r's lock.

   Formalize the requirements below in LTL or CTL.

   (a) It should **not** be possible for a process to lock all three resources.
   Answer: For every process P_i: $\neg\Diamond(has(P_i, r.lock) \wedge has(P_i, s.lock) \wedge has(P_i, t.lock))$

(b) Whenever r is locked, eventually it will be free again.
Answer: $\Box((\exists i : 1{\leq}i{\leq}N : has(P_i, r.lock)) \rightarrow \Diamond\neg(\exists i : 1{\leq}i{\leq}N : has(P_i, r.lock)))$
But this one is just as good: $\Box\Diamond\neg(\exists i : 1{\leq}i{\leq}N : has(P_i, r.lock))$.

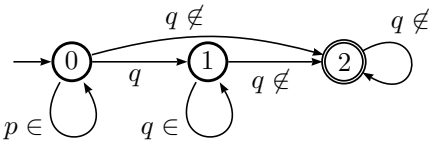
(c) The system should be weakly fair with respect to the resource r. That is, whenever a process P_i requests r's lock, and persistently maintains this request, it should eventually get the lock.
Answer: For every P_i: $\Box(\Box req(P_i, r.lock) \rightarrow \Diamond has(P_i, r.lock))$

(d) When P_i is requesting the resource t, so setting the flag $req(P_i, t.lock)$ to true, it should be possible for P_i to backoff from this request while not getting the resource yet.
Answer. For every P_i:
$\mathbf{AG}(req(P_i, r.lock) \rightarrow \mathbf{E}(req(P_i, r.lock){\wedge}\neg has(P_i, r.lock)\,\mathbf{U}\,\neg req(P_i, r.lock){\wedge}\neg has(P_i, r.lock)))$

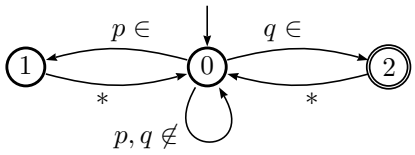
8. [1pt] Give for each LTL formula below, a Buchi automaton that equivalently describes the formula. Do not forget to specify what are the initial and accepting states are. Also, indicate whether you use a standard or generalized Buchi.

Assume $Prop = \{p, q\}$.

(a) $p\,\mathbf{U}\,(q\,\mathbf{U}\,\Box\neg q)$
Answer: Initial state: 0. Accepting state: 2.

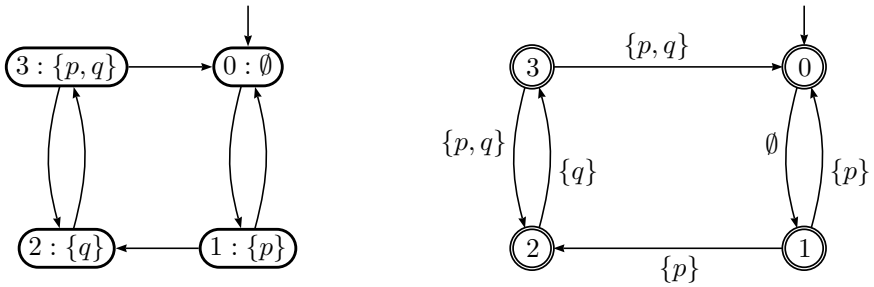


(b) $(\Box\Diamond p) \wedge (\Box\Diamond q)$
Answer: Initial state: 0. Two accepting groups: $\{1\}$ and $\{2\}$.

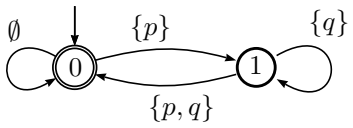


9. [2pt] Consider the following Kripke structure K (left picture) that you can think as modelling some program. $Prop = \{p, q\}$. The initial state is 0.

For your convenience on the right is a Buchi automaton that equivalently describes K (it accepts the same set of the sentences). All its states are marked as accepting.

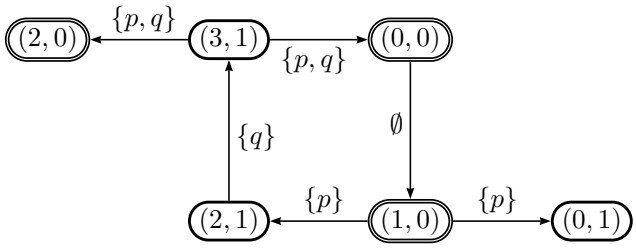


(a) Consider a second Buchi automaton B shown below. The state 0 is the initial state as well as its accepting state.



Construct the intersection automaton $K \cap B$.

Answer: Below is the Buchi automaton $C = K \cap B$. The states of C are pairs (i, j) where i refers to the corresponding state i in K and j the corresponding state j in B. The initial state is $(0, 0)$, and the accepting states are all states of the form $(i, 0)$.



The language of C is btw not empty (it has an accepting run).

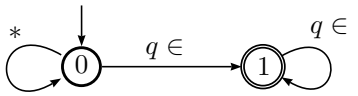
(b) **Describe the steps** of LTL model checking to verify whether the following LTL property ϕ holds on K:

$\Box\Diamond\neg q$

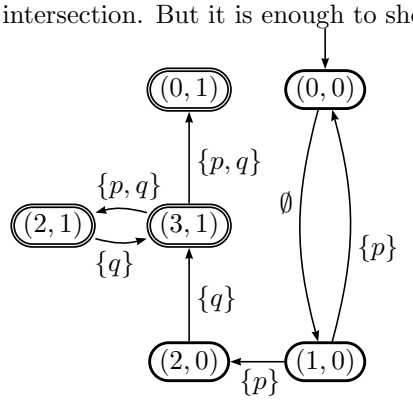
Do **provide your resulting intersection automaton**. You only need to construct it as far as the part that contains a counter example, if there is one. If there is no counter example, you will have to construct the full intersection automaton so that I can check that there is indeed no counter example.

Answer:

i. We first construct a Buchi automaton C representing $\neg\phi(= \Diamond\Box q)$:



ii. Construct the intersection $K_2 = K \cap C$. Below I only show part of the resulting intersection. But it is enough to should a counter example.



iii. Check if the intersection K_2 has an accepting run. If it is then this accepting run is a counter example of ϕ. In the above case K_2 has a reachable accepting state, e.g. $(3, 1)$, which is also part of a cycle $(3, 1) \to (2, 1) \to (3, 1)$. So it has an accepting run.

iv. So, ϕ is not valid on K.

(c) **Describe the steps** of CTL model checking to verify whether the CTL property $\mathbf{AG}(\neg p \vee \neg q)$ holds on K. We can first observe that this property can also be expressed as this, in terms of CTL basic operators:

$$\psi \;=\; \neg\mathbf{E}(\mathbf{true}\ \mathbf{U}\ (p \wedge q))$$

Answer: Let first call the inner formula ψ'. So:

$$\psi \;=\; \neg\underbrace{\mathbf{E}(\mathbf{true}\ \mathbf{U}\ (p \wedge q))}_{\psi'}$$

i. We recursively label the states of K with the sub-formulas of ψ. That is, for every subformula f we calculate the set of states W_f such that $s \in W_f$ if and only if the CTL formula f is valid on $tree(s)$.

ii. We have the following sub-formulas: $true, p, q, p \wedge q$, ψ', and ψ itself. W_{true} is simply all states of K. W_p and W_q are already given in the labelling of K itself. So, $W_p = \{1, 3\}$ and $W_q = \{2, 3\}$.
$W_{p \wedge q} = W_p \cap W_q = \{3\}$.

iii. We calculate $Z = W_{\mathbf{E}(true;\mathbf{U}\ p \wedge q)}$ through fix-point iterations:

A. $Z_0 = W_{p \wedge q} = \{3\}$.

B. Z_{i+1} is Z_i plus any state $s \in W_{true}$ (and not in Z_i, because the latter is added anyway) that has an outgoing arrow/transition to some state $t \in Z_i$. So, $Z_1 = Z_0 \cap \{2\} = \{2, 3\}$

C. Similarly, $Z_2 = Z_1 \cap \{1\} = \{1, 2, 3\}$

D. Similarly, $Z_3 = Z_2 \cap \{0\} = \{1, 2, 3, 4\}$

E. Z_3 contains all states of K, so it must be a fix point. Done. So, $W_{\psi'} = Z = Z_3 = \{0, 1, 2, 3\}$.

iv. $W_\psi = W_{\neg\psi'}$ consists of all states not in $W_{\psi'}$. Since the latter contains all states of K, then W_ψ is empty.

v. The initial state is this not in $W_p si$ we conclude that ψ does not hold on K.

10. [1pt] Let us consider the use of LTL for testing. Consider a non-deterministic program M. Let S be the set of all its possible states. This set can be infinite. The set of all possible executions of M may also be infinite. M has a single intial state, $s_{\mathbf{start}} \in S$, and a single terminal state $S_{\mathbf{end}} \in S$. When M enters $S_{\mathbf{end}}$, it terminates.

A test suite V is a set of tests. Each test produces an execution σ of **finite** length. It may or may not end in $S_{\mathbf{end}}$. Given an LTL formula ϕ, we want to be able to check it using V. We will do it as follows:

(a) For each test $\sigma \in V$, we evaluate $\sigma \models \phi$. However, since σ is finite, we will change the standard definition of $\models$ so that the result is one of these three values:

i. $\sigma \models \phi$ results $\top$ (true) if ϕ holds on all $\sigma +\!\!+ \tau$, where τ is any continuation of σ that M can produce.
Here, $+\!\!+$ is a sequence concatenation operator.

ii. $\sigma \models \phi$ results $\bot$ (false) if ϕ cannot possibly hold on all $\sigma +\!\!+ \tau$, where, as before, τ is any continuation of σ that M can produce.

iii. $\sigma \models \phi$ results *undecided* if you cannot decide if ϕ would or would not hold on all $\sigma +\!\!+ \tau$ (e.g. because ϕ is a future formula and σ is not long enough to decide if ϕ would hold or otherwise).

(b) ϕ is rejected if there is one $\sigma \in V$ where $\sigma \models \phi$ results in $\bot$.
ϕ is accepted if it is not rejected, and there is at least one $\sigma \in V$ where $\sigma \models \phi$ results in $\top$.
Else the result of the verification is undecided.

Your task: give a formal definition of this variation of the evaluation function $\sigma \models \phi$ for LTL formulas. The syntax of LTL is given below. Your definition of $\sigma \models \phi$ should be decidable in finite number of steps.

Hint: define it through $\sigma, i \models \phi$, and recursively over the structure of ϕ.

$$\begin{array}{rcl} \phi & ::= & p \quad \text{, where } p \text{ is a state predicate} \\ & | & \neg\phi \\ & | & \phi \wedge \psi \\ & | & \mathbf{X}\phi \\ & | & \phi \ \mathbf{U} \ \psi \end{array}$$

A 'state predicate' is a predicate that can be evaluated directly on any $s \in S$ to know if it holds there or not.

Answer: Note btw that we cannot evaluate $\sigma \models \phi$ literally by quantifying over all possible continuations of σ as the definition above suggests, since the number of states and the number of executions of M are infinite. Neither does the standard LTL model checking work, due to the infinite number of states.

We define $\sigma \models \phi$ as $\sigma, 0 \models \phi$. The general idea is that $\sigma, i \models \phi$ evaluates the LTL formula ϕ on the suffix $\sigma[i..]$ of σ. How to evaluate $\sigma, i \models \phi$ is shown below.

- $\sigma, i \models p$.
  If $i < |\sigma|$ the meaning is as 'usual'. That is, $\sigma, i \models p$ evaluates to $\top$ if p holds on the state σ_i, and else $\sigma, i \models p$ evaluates to $\bot$.
  If $i > |\sigma$ we would have two cases:
  (a) If σ does not end in $S_{\mathbf{end}}$. So, σ represents an incomplete run. We let $\sigma, i \models p$ evaluate to *undecided* because σ is thus not long enough to make judgement about ϕ (it might hold, with a suitable extension of σ, but we don't know that just from σ itself).
  (b) σ ends in M's' end-state $S_{\mathbf{end}}$. So, $\sigma, i \models p$ tries to evaluate p on some 'state' after M has terminated. The question/assignment is not precise on how this situation should be interpreted. However, a reasonable interpretation is to treat M as if it stutters idefintely after reaching $S_{\mathbf{end}}$. If this interpretation is taken, then evaluating p on σ_i is the same as evaluating p on $S_{\mathbf{end}}$. If it holds there, then $\sigma, i \models p$ evaluates to $\top$, and else $\bot$.
- $\sigma, i \models \neg\phi$ is undecided if $\sigma, i \models \phi$ is undecided. Else it evaluates to the opposite of $\sigma, i \models \phi$. So, $\sigma, i \models \neg\phi$ is $\top$ if $\sigma, i \models \phi$ is $\bot$, and else it is $\bot$.
- $\sigma, i \models \phi \wedge \psi$ is undecided if both $\sigma, i \models \phi$ and $\sigma, i \models \psi$ are undecided. Else, if both $\sigma, i \models \phi$ and $\sigma, i \models \psi$ are $\top$, then $\sigma, i \models \phi \wedge \psi$ evaluates to $\top$
  Else, then either $i, \sigma \models \phi$ or $\sigma, i \models \psi$ is $\bot$ (the other might be $\top$ or even undecided), and then $\sigma, i \models \phi \wedge \psi$ evaluates to $\bot$ too.
- $\sigma, i \models \mathbf{X}\phi \quad = \quad \sigma, i{+}1 \models \phi$.
- $\sigma, i \models \phi\mathbf{U}\psi$ evaluates to $\top$ if there is a k, $i{\leq}k$ such that $\sigma, k \models \psi = \top$, and for all j, $i{\leq}j{<}k$, we have $\sigma, k \models \phi = \top$.
  Else, if there is a k, $i{\leq}k$ such that $\sigma, k \models \neg\phi \wedge \neg\psi = \top$, and for all j, $i{\leq}j{<}k$, we have $\sigma, k \models \phi \wedge \neg\psi = \top$, then $\sigma, i \models \phi\mathbf{U}\psi$ evaluates to $\bot$.
  For other cases, $\sigma, i \models \phi\mathbf{U}\psi$ is undecided.